IS2203 Object-Oriented Programming

Practical - 01

ගමන්සර (GamanSara): Transforming Sri Lanka's Public Transport System

In recent years, Sri Lanka has faced increasing challenges in managing its public transportation system, especially in densely populated urban regions such as Colombo and its surrounding suburbs. With thousands of daily commuters relying on buses and trains for work, education, and daily activities, the need for a more efficient and coordinated system has become essential.

To address these challenges, the Ministry of Transport introduced a digital initiative called "ගමන්සර (GamanSara)", a centralized transport management system designed to monitor and manage public buses and railway services across the country.

During its initial phase, GamanSara was developed as a procedural system using the C programming language. The development team focused on building a fast and straightforward solution that could handle basic operations such as ticket pricing, bus scheduling, route tracking, and passenger logging.

The system consisted of several independent functions such as calculateFare, assignBus, printTicket, and updateRouteDetails. Data related to buses, passengers, and routes were defined using structures and passed between functions whenever needed.

At the beginning, the system performed reasonably well. However, as the transportation network expanded across major regions, including Kandy, Matale, Galle, and Jaffna, the limitations of the procedural approach became increasingly visible.

The Sri Lanka Transport Board (SLTB) operates a large fleet of buses, each with different seating capacities, service types (normal, semi-luxury, and luxury), and fare rules. At the same time, Sri Lanka Railways manages multiple train categories, such as express, intercity, and night mail trains. Each of these services requires different handling in terms of scheduling, pricing, and passenger management.

To accommodate these differences, developers repeatedly modified existing functions. The calculateFare function, for example, became overloaded with conditional logic to handle various vehicle types, distances, and passenger categories. Similarly, scheduling functions had to be rewritten whenever new routes or special services were introduced during peak seasons or holidays.

Over time, the codebase grew complex and difficult to manage. Even a minor update—such as introducing a discounted fare for students—required changes across multiple functions. This often resulted in unexpected errors in other parts of the system.

Another major issue was data inconsistency and lack of control. Since data structures were accessed directly by multiple functions, there were cases where critical information such as ticket prices or passenger details was modified incorrectly. In one instance, a pricing update for a specific route accidentally affected fares across multiple districts, causing confusion among passengers.

Moreover, the system lacked a clear representation of real-world entities. A “bus” or a “train” was not treated as a complete unit with its own properties and behaviors. Instead, data and related operations were scattered across different parts of the program, making it difficult to understand how the system worked as a whole.

As Sri Lanka moved towards digital transformation with mobile ticketing apps, GPS-based vehicle tracking, and online reservation systems, the limitations of the existing procedural system became a serious concern. The system was not flexible enough to integrate new technologies or scale efficiently.

Recognizing these challenges, the development team proposed migrating GamanSara to an Object-Oriented Programming (OOP) approach using C++.

In the new design, the system would model real-world entities as objects:

A Bus would include details such as route number, capacity, and service type, along with functions like assignRoute and calculateFare.

A Train would have attributes like coach count, schedule, and travel class, with methods to manage reservations and timing.

A Passenger would store personal details and travel history, along with functions such as bookTicket and viewJourney.

By grouping data and functions together, the system would ensure better control and reduce the risk of unauthorized data modification. The introduction of classes would allow developers to create reusable and organized code structures.

Furthermore, OOP concepts such as encapsulation would help protect sensitive data, while abstraction would allow the system to hide unnecessary complexity from users and developers. This would make the system easier to maintain and extend.

The transition to C++ would also introduce powerful features such as improved data handling, modular programming, and support for object-oriented design principles. This shift would enable GamanSara to evolve into a scalable and robust system capable of supporting future innovations in Sri Lanka’s transportation sector.

At present, the development team is in the early stages of this transformation. Before implementing the new system, they must analyze the limitations of the current approach and understand how object-oriented principles can be applied effectively.

The success of GamanSara depends on this crucial transition—from a fragmented procedural system to a well-structured, object-oriented solution that reflects the real-world complexity of Sri Lanka's transport network.

1. Write both C and C++ Basics.
    A. Print a welcome message for the system

    ```
    Welcome to GamanSara Transport System
    ```

    B. Prompt the user to input the number of passengers waiting at a bus stop.
2. Write a **C++ program** to display the following:

    ```
    GamanSara is a Smart Transport Management System.
    It will be developed using Object Oriented Programming.
    ```

3. Write a C++ program to:
   Declare:
    - An integer variable → busCount = 20
    - An integer variable → trainCount = 18
    - A float variable → averageFare = 14.2

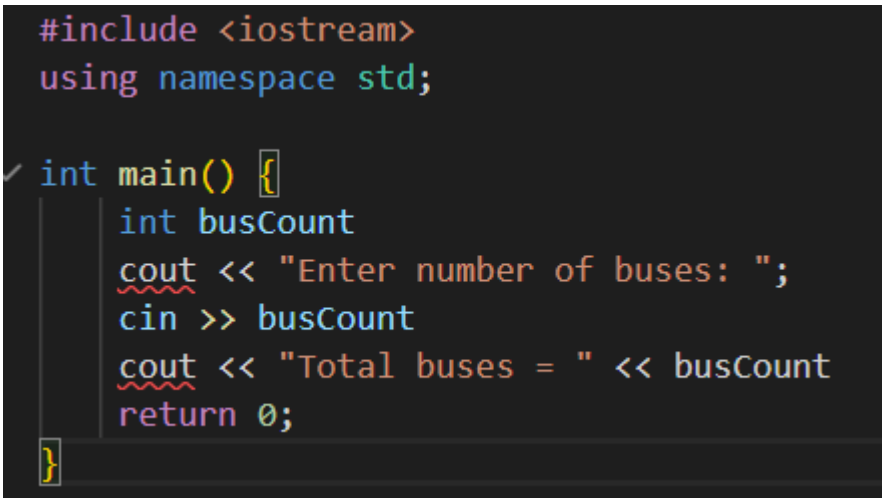
   Display all values clearly.
4. Write a **C++ program** to input and display passenger details:
    - Name
    - Age
    - Gender (M/F)
    - Height

   Display output like:

    ```
    Enter Name: Sathsara
    Enter Age: 22
    Enter Gender (M/F): M
    Enter Height: 164

    Passenger Details:
    Name: Sathsara
    Age: 22
    Gender: M
    Height: 164
    ```

5. Identify errors, correct, and rewrite the program:



```cpp
#include <iostream>
using namespace std;

int main() {
    int busCount
    cout << "Enter number of buses: ";
    cin >> busCount
    cout << "Total buses = " << busCount
    return 0;
}
```

6. What are the limitations of C (procedural programming)?
7. Why is C++ chosen for implementing OOP in this system?
8. In which situations could procedural programming still be useful?